

Information Retrieval 11

Learned Sparse Retrieval

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Bi-encoder: Independently vectorizes queries and documents, scoring by dot product or cosine similarity. Document vectors can be pre-computed for fast retrieval

DPR: BERT Bi-encoder + training with in-batch negatives. Significantly outperforms BM25 in Open-domain QA

ColBERT: Token-level Late Interaction (MaxSim), achieving both the efficiency of Bi-encoders and accuracy close to Cross-encoders

This time we learn **Learned Sparse Retrieval**, the opposite paradigm to Dense Retrieval. We will understand the mechanism of SPLADE in detail and cover **hybrid search** that combines Sparse and Dense

Dense Retrieval is powerful, but sparse representations have unique advantages. SPLADE leverages BERT's knowledge to learn sparse representations, achieving semantic search while maintaining the efficiency of inverted indexes

First Half: Learned Sparse Retrieval

- Design space of retrieval paradigms
- SPLADE: Learned sparse representations
- SPLADEv2 / SPLADE++ improvements
- Detailed comparison of Sparse vs Dense

Second Half: Hybrid Search and Summary

- Hybrid search methods
- Reciprocal Rank Fusion and learned fusion
- Review of all 11 lectures
- Future directions

Design Space of Retrieval Paradigms

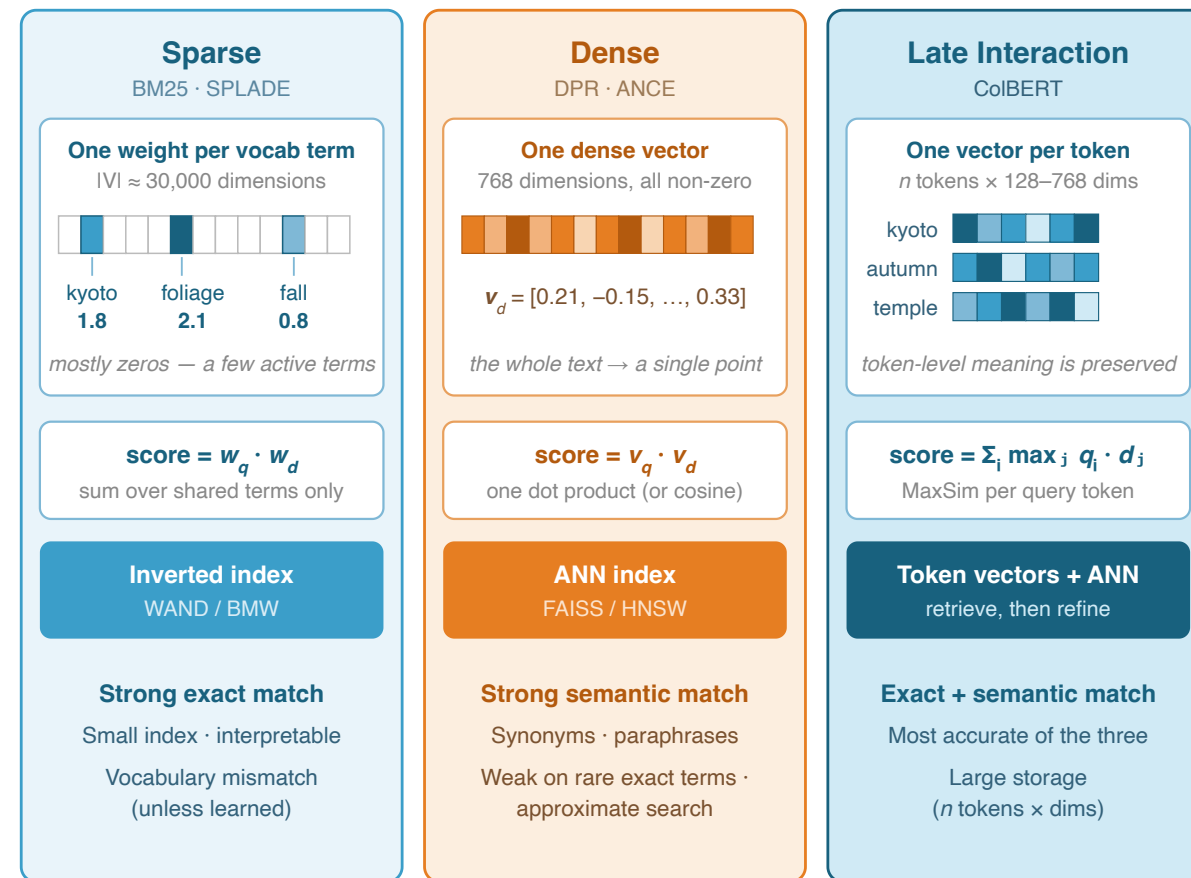
Sparse, Dense, and Late Interaction

The retrieval methods covered in previous lectures can be organized by representation **density** and **interaction granularity**

Sparse Retrieval (Lectures 1, 9): Independent weight per term (BM25, DeepCT, DeepImpact). Fast retrieval via inverted index, but score is 0 if vocabulary does not match

Dense Retrieval (Lecture 10): A single dense vector per query/document (DPR, ANCE). Fast retrieval via ANN. Captures semantic similarity but weak at exact matching

Late Interaction (Lecture 10): Sets of token-level dense vectors (ColBERT). Fine-grained interactions via MaxSim



Property	Sparse	Dense	Late Interaction
Representation	Sparse vector over vocabulary	Fixed-dimension dense vector	Token-level dense vector sets
Index	Inverted index	ANN (FAISS, HNSW)	Token vectors + ANN
Exact matching	Strong	Weak	Strong
Semantic matching	Weak (traditional) / Possible (learned)	Strong	Strong
Storage	Small (sparse)	Medium (vectors)	Large (num tokens x dimensions)
Latency	Low	Low to Medium	Medium
Representative methods	BM25, SPLADE	DPR, ANCE	ColBERT

Question: Can we maintain Sparse Retrieval's strength in exact matching while also achieving semantic matching like Dense Retrieval? -> This is the motivation for **Learned Sparse Retrieval**

Limitations of DeepCT and DeepImpact (review from Lecture 9):

DeepCT: Reweights only **terms already present in the document**

DeepImpact: Learns weights after expansion with doc2query. Complex two-stage pipeline

Ideal Learned Sparse model:

1. **Vocabulary expansion:** Add related terms absent from the document
2. **Importance estimation:** Learn term weights based on context
3. **End-to-End learning:** Expansion and weighting in a single model
4. **Inverted index compatible:** Use existing infrastructure as-is

DeepCT vs Ideal Model

DeepCT: Only reweighting existing terms

Input: "Beautiful **autumn leaves temples** in Kyoto"

Output: Kyoto=**3**, autumn_leaves=**5**, beautiful=**0**, temples=**4**

Problem: "fall" and "tourism" are not added

SPLADE: Expansion + weighting simultaneously

Input: "Beautiful autumn leaves temples in Kyoto"

Output: Kyoto=**3**, autumn_leaves=**5**, temples=**4**,

fall=2, tourism=1, foliage=2

-> Resolves vocabulary mismatch

SPLADE

Sparse Lexical And Expansion Model

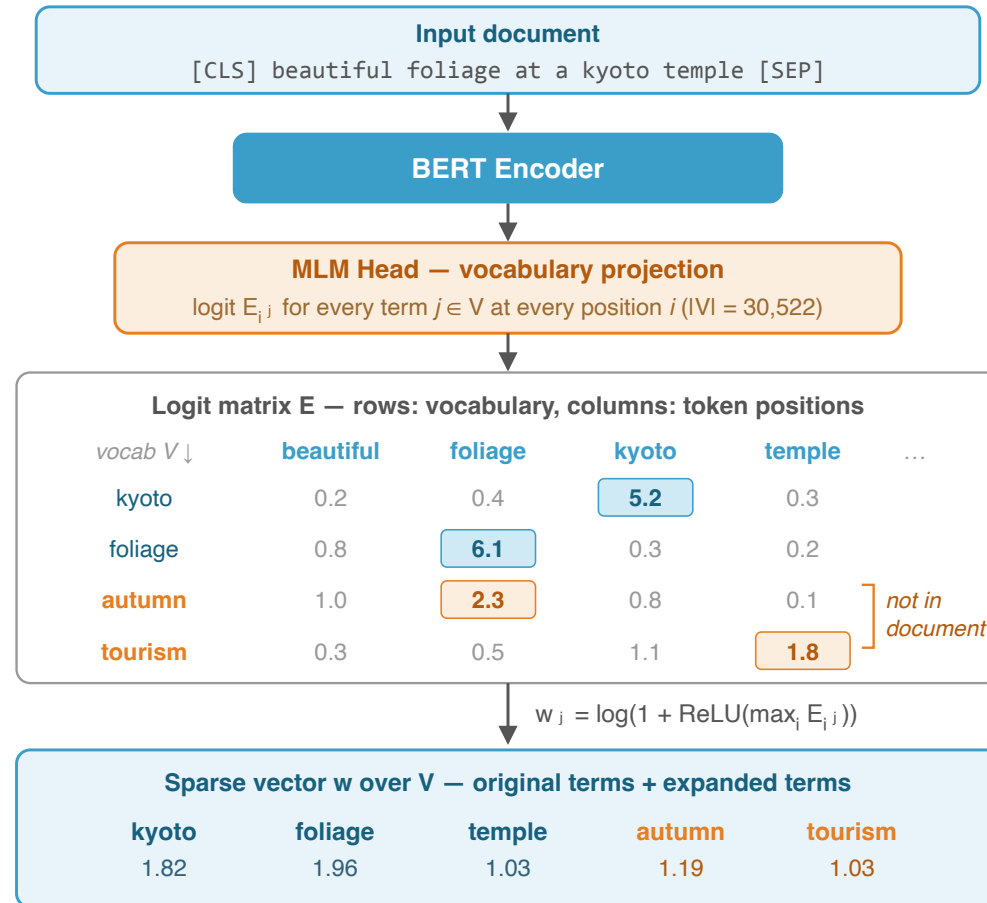
SPLADE (SParse Lexical AnD Expansion model)

[Formal et al., 2021] learns sparse representations by leveraging BERT's **Masked Language Model (MLM) head**

Insight: The MLM head predicts "which term should appear at a given token position" — essentially the same capability as **vocabulary expansion**

Given "beautiful [MASK] in Kyoto", BERT can predict "leaves", "cherry blossoms", "gardens"; this capability is repurposed for **vocabulary expansion for retrieval**

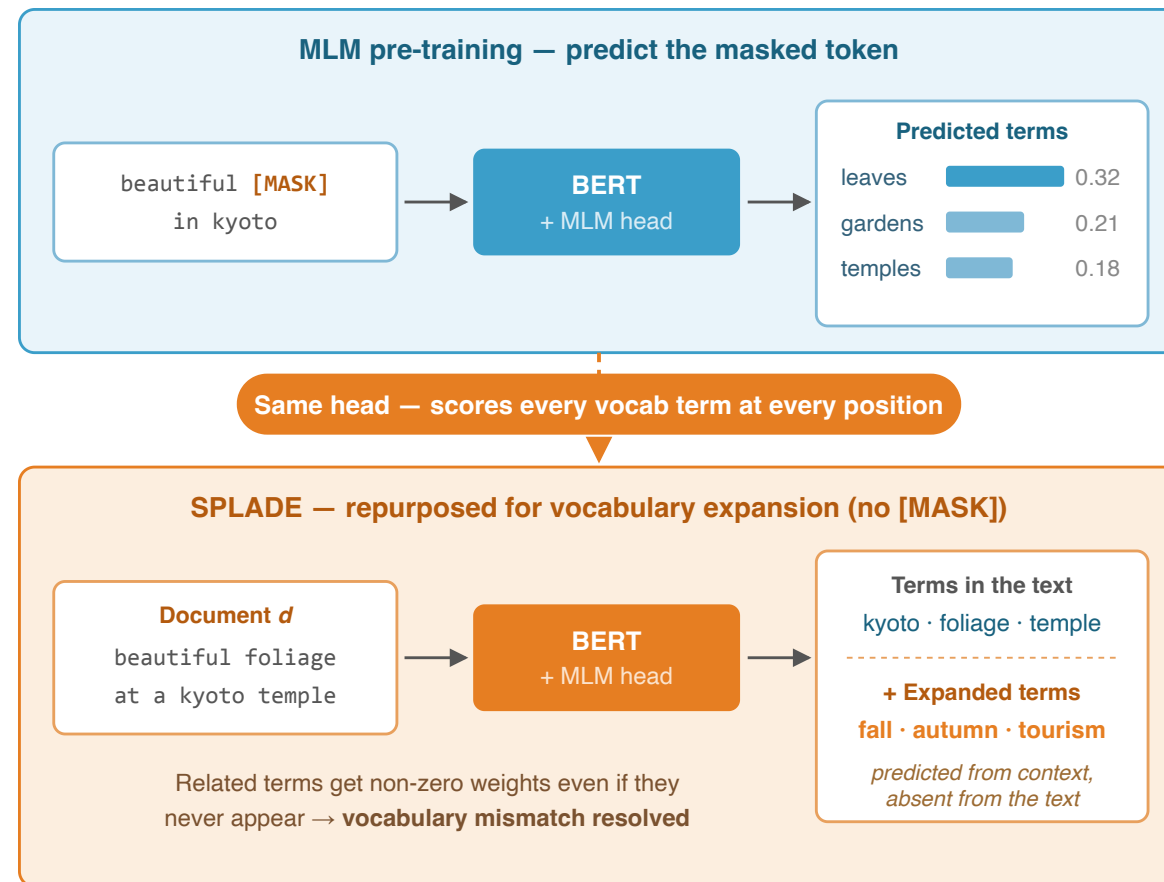
At each token position, compute an importance distribution over the entire vocabulary, then aggregate per term into a sparse representation



How the MLM head enables vocabulary expansion:

1. For each token position in the document, BERT + MLM head outputs importance scores over the **entire vocabulary**
2. For example, at the position "autumn leaves", high scores are assigned to **related terms not in the document** such as "fall", "tourism", "temple"
3. Take the maximum value across all positions for each term (max-pooling)
4. This results in a sparse representation of original terms + expanded terms

log-saturation: The transformation $\log(1 + \text{ReLU}(E_{ij}))$ achieves an effect similar to BM25's TF saturation



For each token position i in document d , BERT + MLM head yields **logits** E_{ij} over the entire vocabulary V

Importance of each term j :

$$w_j = \max_{i \in d} \log(1 + \text{ReLU}(E_{ij}))$$

E_{ij} : MLM logit for term j at position i

ReLU: Clips negative values to 0

$\log(1 + \cdot)$: Logarithmic transformation similar to TF saturation

max: Adopts the maximum importance across all positions

Intuition: If term j is strongly predicted at any position in the document, then that term is an important term representing the document's content

Example: Per-position Logits and Aggregation

Position	Kyoto	autumn_leaves	fall	tourism
pos 1 "Kyoto"	5.2	0.3	0.8	1.1
pos 2 "no"	0.1	0.1	0.0	0.0
pos 3 "autumn_leaves"	0.4	6.1	2.3	0.5
pos 4 "ga"	0.0	0.1	0.1	0.0
pos 5 "beautiful"	0.2	0.8	1.0	0.3
pos 6 "temple"	0.3	0.2	0.1	1.8

After max-pooling: Kyoto= $\log(1+5.2)=$ **1.82**,
autumn_leaves= $\log(1+6.1)=$ **1.96**, fall= $\log(1+2.3)=$ **1.19**,
tourism= $\log(1+1.8)=$ **1.03**

"fall" and "tourism" do not appear in the document, but the MLM head predicts them from context -> **Vocabulary expansion** is achieved

Scoring: Dot product of sparse representations of query q and document d

$$\text{score}(q, d) = \sum_{j \in V} w_j^q \cdot w_j^d$$

The query is also encoded by BERT; only dimensions where vocabulary matches are non-zero -> efficiently computed with inverted index

Training: Ranking loss (negative log-likelihood) with relevant d^+ and non-relevant d^- (BM25 hard negatives + in-batch negatives):

$$\mathcal{L}_{\text{rank}} = -\log \frac{\exp(s(q, d^+))}{\exp(s(q, d^+)) + \sum_{d^- \in \mathcal{N}} \exp(s(q, d^-))}$$

Compatibility with Inverted Index

SPLADE output — sparse vectors

Query q : { kyoto 1.5, foliage 2.0, **fall 0.8** }

Doc d_1 : { kyoto 1.8, foliage 2.1, temple 1.5, **fall 1.2, tourism 1.0** }

each term–weight pair is stored as a posting

Inverted index — same structure as BM25

kyoto → (d_1 , 1.8) (d_5 , 2.3) (d_9 , 1.1) ...

foliage → (d_1 , 2.1) (d_3 , 1.5) ...

temple → (d_1 , 1.5) (d_8 , 2.0) ...

tourism → (d_1 , 1.0) (d_2 , 3.4) (d_5 , 2.3) ...

fall → (d_1 , 1.2) (d_7 , 0.9) ...

← expanded term

"fall" never appears in d_1 's text, yet d_1 is now retrievable by it

Weights come from SPLADE instead of TF-IDF statistics — everything else is unchanged

The same infrastructure as BM25 (inverted index + fast retrieval algorithms such as WAND) can be used as-is. This is a major practical advantage over Dense Retrieval

Problem: Without constraints, the MLM head spreads weights across the entire vocabulary and the representation becomes **dense**, losing the efficiency of inverted indexes

FLOPS regularization (FLoating point Operations Per Second):

$$\mathcal{L}_{\text{FLOPS}} = \sum_{j \in V} \bar{a}_j^2, \quad \bar{a}_j = \frac{1}{N} \sum_{i=1}^N w_j^{(i)}$$

$\bar{a}_j$: Average activation of term j , over the N documents in the batch

Squaring strongly penalizes terms activated in many documents (= long posting lists)

Intuition: Similar effect to IDF. Suppresses weights of frequent terms, retaining discriminative terms

Effect of FLOPS Regularization

Without regularization

Out of 30,000 vocabulary terms, average **5,000 terms** are non-zero

-> Inverted index becomes enormous

-> Search latency is 10x or more than BM25

With FLOPS regularization

Out of 30,000 vocabulary terms, average **100-300 terms** are non-zero

-> Index size close to BM25

-> High accuracy at BM25-equivalent latency

Final loss function:

$$\mathcal{L} = \mathcal{L}_{\text{rank}} + \lambda_q \mathcal{L}_{\text{FLOPS}}^q + \lambda_d \mathcal{L}_{\text{FLOPS}}^d$$

FLOPS regularization is applied to both query and document sides. Hyperparameter *lambda* controls the trade-off between sparsity and accuracy

SPLADEv2 [Formal et al., 2022] improvements:

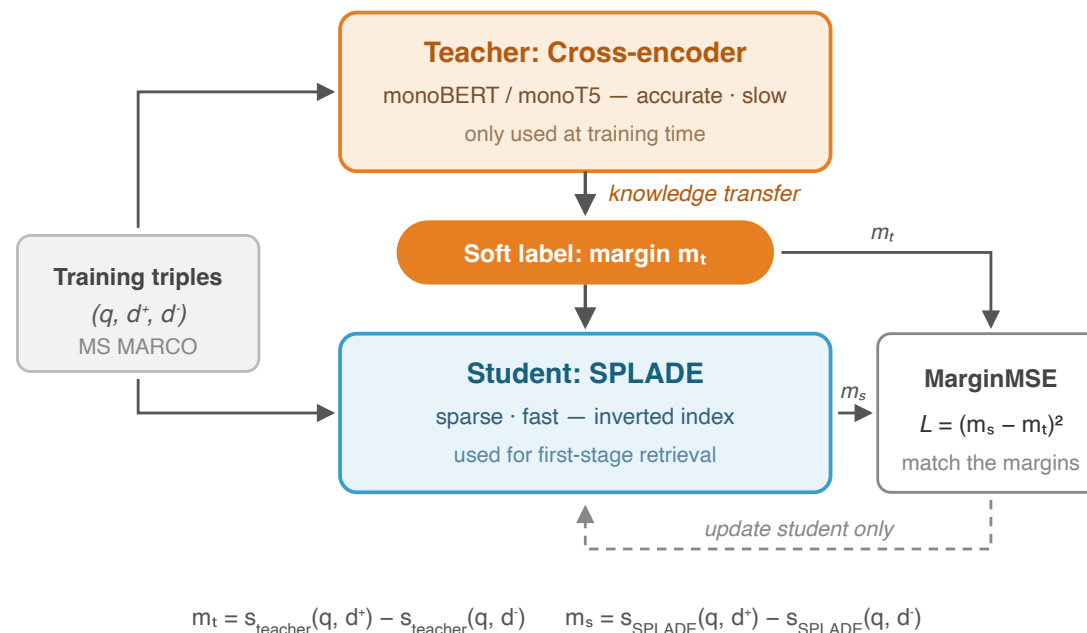
1. **Distillation-based training:** Distills high-accuracy Cross-encoder scores into the sparse model with MarginMSE loss $\mathcal{L} = (m_s - m_t)^2$ (m_s : student margin, m_t : teacher margin)
2. **Improved hard negatives:** Uses Dense Retriever negatives in addition to BM25 negatives

SPLADE++ further improvements:

Self-Distillation: Feeds SPLADE's own output back as teacher signal

Ensemble Distillation: Distills from an ensemble of multiple Cross-encoders — SPLADE++ EnsembleDistil achieves the best performance

Conceptual Diagram of Distillation



Transfers Cross-encoder accuracy to SPLADE while maintaining SPLADE's sparsity and search efficiency. Achieves the "best of both worlds" in accuracy and efficiency

MRR@10 on MS MARCO passage ranking dev set:

Model	Type	MRR@10
BM25	Sparse	0.187
DeepCT	Learned Sparse	0.243
DeepImpact	Learned Sparse	0.326
SPLADE	Learned Sparse	0.340
DPR	Dense	0.311
ANCE	Dense	0.330
ColBERT	Late Int.	0.360
SPLADEv2	Learned Sparse	0.368
SPLADE++ ED	Learned Sparse	0.380

SPLADE++ ED = SPLADE++ EnsembleDistil. Numbers from Formal et al. (2021, 2022)

Formal et al., SIGIR, 2021; Formal et al., SIGIR, 2022; Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering", EMNLP, 2020; Khattab and Zaharia, "ColBERT", SIGIR, 2020

Key Points

1. **SPLADE outperforms DPR:** Demonstrates that a sparse model can beat Dense Retrieval
2. **Large effect of distillation:** MRR improves by 0.028 from SPLADE to SPLADEv2. Cross-encoder knowledge transfers effectively
3. **SPLADE++ rivals ColBERT:** Achieves accuracy close to Late Interaction using an inverted index

Note: MRR@10 values vary depending on training data, negative sampling, and use of distillation. Fair comparison requires experiments under identical conditions. The above are reported values from each paper, and conditions are not fully unified

Aspect	doc2query	DeepCT	DeepImpact	SPLADE
Vocabulary expansion	Generated by T5	None	Depends on doc2query	Implicit via MLM
Weight learning	None (BM25)	Learned by BERT	Learned by BERT	Learned by BERT
End-to-End	Two stages	Single stage (weights only)	Two stages	Single stage
Regularization	None	None	None	FLOPS
Index	Inverted (expanded)	Inverted	Inverted (expanded)	Inverted (expanded)
MS MARCO MRR@10	0.272	0.243	0.326	0.368

SPLADE can be positioned as a unified evolution of doc2query, DeepCT, and DeepImpact. It achieves vocabulary expansion and weight learning End-to-End, with FLOPS regularization enabling efficiency control

Approximate values. Formal et al., SIGIR, 2021; 2022; Nogueira and Lin, 2019; Dai and Callan, 2019; Mallia et al., 2021

Sparse vs Dense Comparison

When to Use Which?

Strengths of Sparse Retrieval (BM25 / SPLADE):

Exact matching strength: Strong for exact-match queries with proper nouns, model numbers, IDs, etc.

e.g., "iPhone 15 Pro Max" retrieves documents where terms match with high recall

Interpretability: Clear which terms matched and contributed to the score

Mature infrastructure: Inverted indexes have decades of optimization (WAND, BMW, etc.)

Zero-shot generalization: BM25 provides consistent performance regardless of domain

Weaknesses:

Vocabulary mismatch problem (BM25) ->

Mitigated by SPLADE

BM25 term weights are statistical (not learnable)

Strengths of Dense Retrieval (DPR / ANCE):

Semantic matching: Naturally captures synonyms and paraphrases

e.g., "dog walking" -> Can retrieve "pet exercise"

Multilingual support: Cross-lingual retrieval possible with multilingual models

Representational flexibility: Easy extension to non-text modalities

Weaknesses:

Weak at exact matching (tends to fail on rare proper nouns)

ANN indexes are approximate, recall is not 100%

Performance may degrade significantly on out-of-domain data

Large vector storage

Hybrid Search

Combining Sparse and Dense

Sparse and Dense are complementary, as empirically demonstrated:

There are many queries where BM25 returns correct results but DPR fails, and vice versa

Combining both achieves higher performance than either alone

Basic design:

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot s_{\text{sparse}}(q, d) + (1 - \alpha) \cdot s_{\text{dense}}(q, d)$$

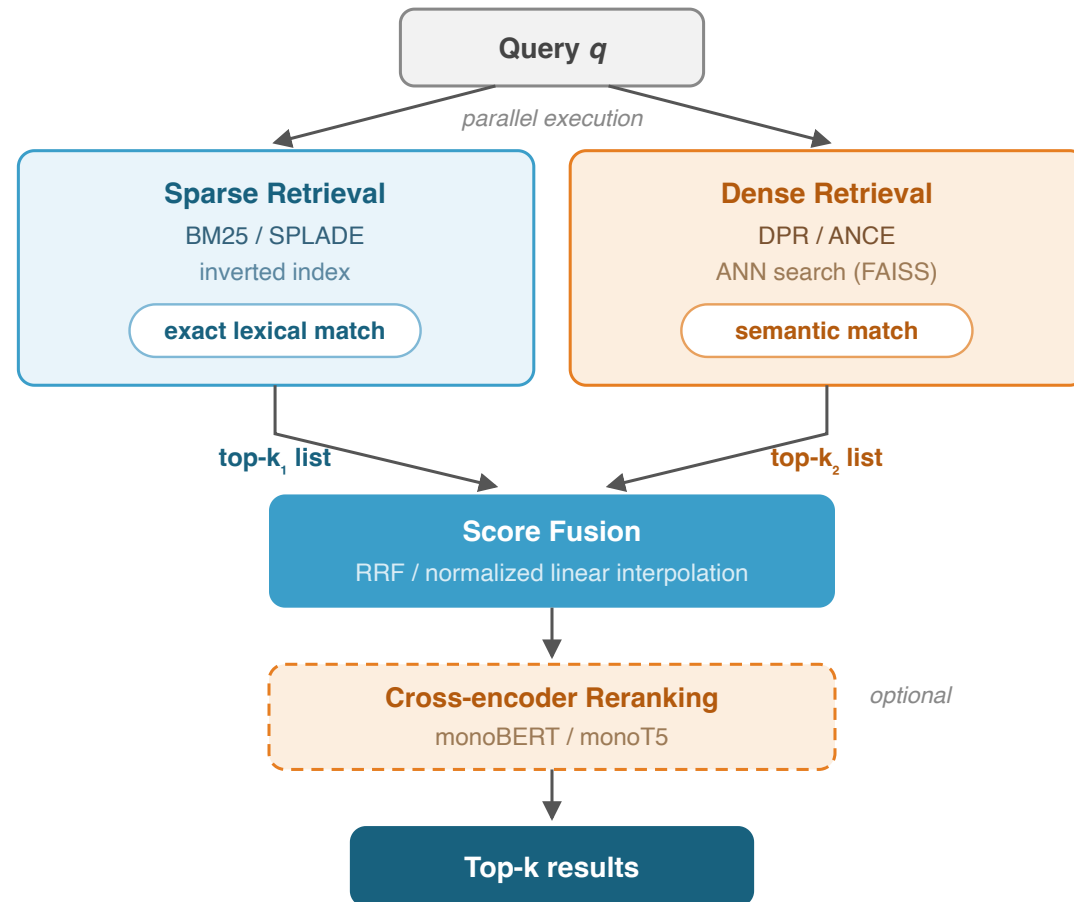
$\alpha \in [0, 1]$: Weight between sparse and dense

Problem: Score scales differ -> **Normalization** is needed

Challenges of linear interpolation:

Score distributions differ (BM25: 0-40, DPR: 0-1)

Min-max normalization or rank-based fusion is effective



RRF [Cormack et al., 2009] is a rank-based fusion method that eliminates the need for score normalization

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

R : Set of rankings to be fused

$\text{rank}_r(d)$: Rank of document d in ranking r

k : Constant (typically 60). Mitigates excessive concentration on top ranks

Advantages:

Independent of score scale

Simple with only hyperparameter k

Can fuse three or more rankings

No training required

Widely adopted in practice: Widely used in commercial search systems such as Elasticsearch

RRF Calculation Example

With $k = 60$:

Doc	BM25 Rank	DPR Rank	RRF Score
d1	1	3	$1/61 + 1/63 = \mathbf{0.032}$
d2	3	1	$1/63 + 1/61 = \mathbf{0.032}$
d3	2	5	$1/62 + 1/65 = \mathbf{0.032}$
d4	10	2	$1/70 + 1/62 = \mathbf{0.030}$
d5	4	50	$1/64 + 1/110 = \mathbf{0.025}$

d1 and d2: Rank 1 in one system, rank 3 in the other -> Same RRF score.
d5: Rank 4 in BM25 but rank 50 in DPR -> Low score (one system alone is insufficient)

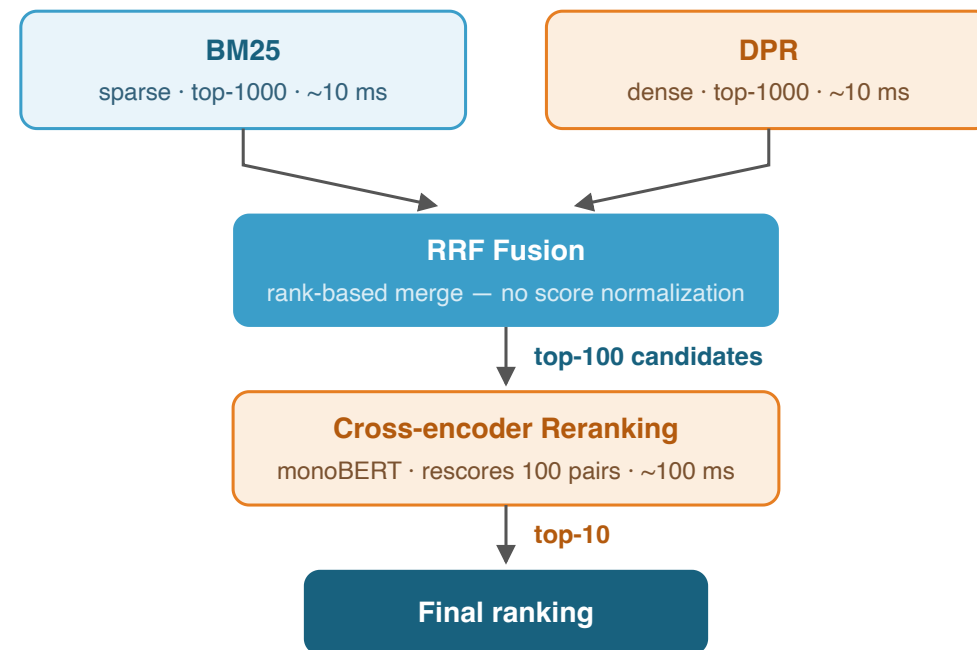
Documents ranked **highly in both systems** receive high RRF scores

Limitations of RRF: Treats all rankings equally and cannot adapt per query

Learning-based fusion methods:

1. **Learning linear interpolation weights:** Optimize α on validation data. Simple but fixed weight independent of the query
2. **Query-dependent weighting:** Vary α based on query features (length, ambiguity, ...). Short queries -> emphasize Dense, long queries -> emphasize Sparse
3. **Cross-encoder reranking:** Merge Sparse + Dense candidates, then rerank with a Cross-encoder. Highest accuracy but high computation cost

Practical Pipeline



Different models are used at each stage of the multi-stage pipeline. First stage prioritizes speed (Sparse/Dense), second stage prioritizes accuracy (Cross-encoder)

Standard inverted index (BM25):

```
"Kyoto" -> [(d1, 3.2), (d5, 2.1), (d9, 1.8)]  
"autumn_leaves" -> [(d1, 4.1), (d3, 3.5)]  
"tourism" -> [(d2, 2.8), (d5, 1.9)]
```

Weights = per-term components of BM25 score

Only terms appearing in the document have entries

SPLADE inverted index:

```
"Kyoto" -> [(d1, 5.2), (d5, 3.1), (d9, 2.4)]  
"autumn_leaves" -> [(d1, 4.8), (d3, 3.9)]  
"tourism" -> [(d1, 1.2), (d2, 3.4), (d5, 2.3)]  
"fall" -> [(d1, 1.9), (d3, 1.4)] <- expanded
```

Differences:

In SPLADE, **terms not appearing in the document** also have entries (vocabulary expansion)

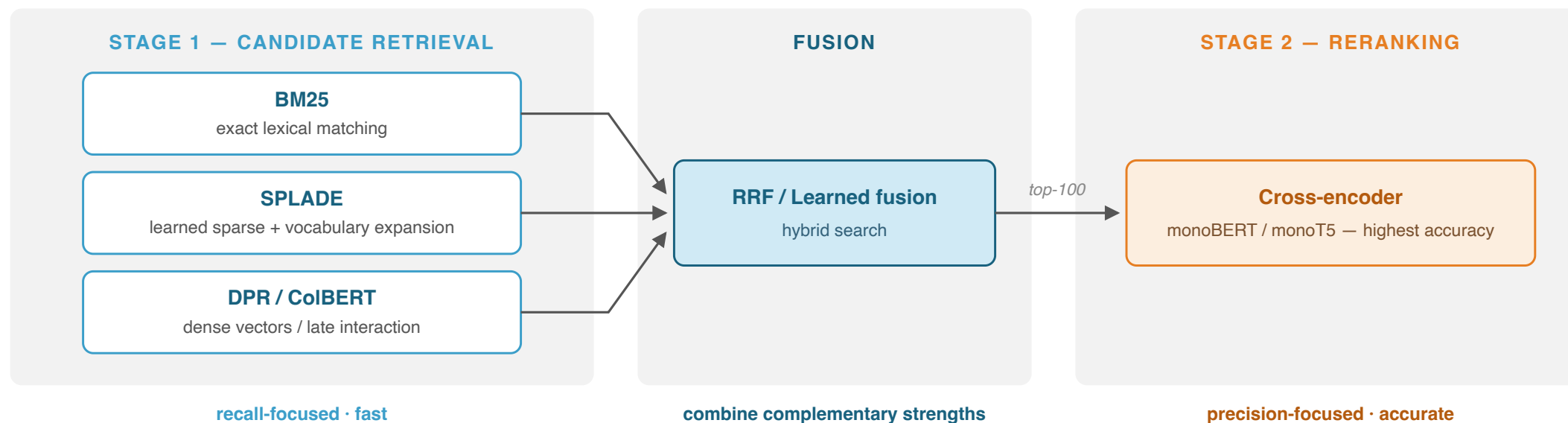
Weights are learned values computed by BERT + MLM head

Posting list lengths increase -> **Controlled by FLOPS regularization**

Computation at search time

Score = Traverse posting lists of query terms and accumulate scores per document. **Exactly the same algorithms** (WAND, BMW, etc.) as BM25 can be used

$$s(q, d) = \sum_{t \in q} w_t^q \cdot w_t^d$$



Sparse (BM25, SPLADE): Reliability of exact matching, efficiency, interpretability

Dense (DPR, ColBERT): Semantic matching, multilingual support

Hybrid: Integrates the strengths of both

Cross-encoder: Highest accuracy reranking at the final stage

Learned Sparse Retrieval (SPLADE) achieved both practicality and accuracy by incorporating neural model semantic understanding while maintaining the traditional BM25 inverted index infrastructure

1. **Learned Sparse Retrieval:** A paradigm that performs vocabulary expansion and weight learning with neural models while maintaining the advantages of BM25's sparse representation (efficiency, interpretability, existing infrastructure compatibility)
2. **SPLADE:** Leverages BERT's MLM head to simultaneously achieve vocabulary expansion and importance estimation. Controls sparsity with FLOPS regularization, enabling continuous adjustment of the accuracy-efficiency trade-off
3. **SPLADEv2 / SPLADE++:** Significantly improves performance through knowledge distillation and Hard Negative Mining. Also surpasses Dense Retrieval in zero-shot generalization
4. **Hybrid search:** Combines Sparse (exact matching) and Dense (semantic matching) via RRF, etc., achieving higher performance than either alone

Preview of Next Lecture

Connection to content from Lecture 13 onward